

Documento Técnico de Diseño

Arquitectura Híbrida de Base de Datos para Ecommify: Diseño Conceptual y Lógico

David Ricardo Grandas Cárdenas

Danilo Andrés Cortés Saavedra

Edisson Steven Bustos Galeano

Universidad de La Sabana

Diseño y Optimización de Base de Datos

Miguel Alfonso Varela

Mayo 2026

1. Resumen Ejecutivo

1.1. Información del Equipo

Integrante	Rol en el Proyecto	Fortalezas Técnicas
David Ricardo Grandas Cárdenas	Líder técnico de bases de datos	PostgreSQL, modelado relacional, normalización
Danilo Andrés Cortés Saavedra	Diseñador NoSQL y analítica	MongoDB, Python/pandas, análisis exploratorio
Edisson Steven Bustos Galeano	Documentación e integración	Redacción técnica, Google Colab, ETL

1.2. Enfoque Arquitectónico Seleccionado

El equipo seleccionó la Opción 3: Arquitectura Políglota Híbrida, que combina PostgreSQL en Supabase para el módulo transaccional y MongoDB en Atlas M0 para el módulo analítico. Esta decisión se fundamenta en los patrones de acceso y la naturaleza de los datos identificados en el Análisis Exploratorio de Datos (EDA) sobre el dataset Brazilian E-commerce (Olist).

1.3. Abstract

Este documento presenta el diseño conceptual y lógico completo de la arquitectura híbrida de base de datos para Ecommify, una plataforma de comercio electrónico multivendedor especializada en productos tecnológicos. La solución integra PostgreSQL (Supabase) para datos transaccionales y MongoDB (Atlas M0) para datos analíticos, siguiendo los principios del Teorema CAP aplicados a cada módulo del sistema. El diseño relacional cumple la Tercera Forma Normal (3FN) en todas las entidades transaccionales, con aplicación justificada de tipos avanzados de PostgreSQL: JSONB para especificaciones de productos, TEXT[] para galerías de imágenes y etiquetas, TSTZRANGE para períodos de promoción y composite types para direcciones. Se documenta la estrategia de particionamiento de la tabla orders por rango de fechas, vistas materializadas para dashboards analíticos, triggers de auditoría y plan de mantenimiento operativo. En el módulo NoSQL se definen cuatro colecciones en MongoDB con patrones de modelado basados en los principios de embebido versus referencia, incluyendo un índice TTL de 30 días para comportamiento de usuario. Las decisiones arquitectónicas se sustentan en una matriz de 9 módulos con criterios de volumen, consistencia, estructura y patrones de acceso, validados con datos reales del dataset Olist (99.441 pedidos, 32.951 productos, 96.096 clientes).

2. Análisis de Requisitos

2.1. Contexto de Ecommify

Ecommify es una plataforma de comercio electrónico multivendedor orientada a productos tecnológicos. El sistema gestiona dos dominios de datos claramente diferenciados: (1) datos transaccionales con alta exigencia de consistencia ACID (pedidos, pagos, inventario, usuarios) y (2) datos analíticos con esquemas flexibles y alta variabilidad (catálogo de productos, reseñas, comportamiento de navegación). La base de datos utilizada para el proyecto académico es el conjunto de datos real Brazilian E-commerce Public Dataset by Olist, disponible en Kaggle.

2.2. Requisitos Funcionales

- Gestión de usuarios: registro, autenticación JWT, roles (comprador/vendedor/admin), recuperación de contraseña.
- Catálogo de productos: listado, filtrado por categoría y precio, búsqueda tolerante a errores tipográficos.
- Carrito de compras: persistencia de sesión, cálculo de totales, validación de stock en tiempo real.
- Checkout y pagos: flujo en 3 pasos, múltiples métodos de pago, confirmación y notificación.
- Sistema de reseñas: publicación verificada (solo compradores), calificación promedio automática.
- Panel de vendedores: gestión de stock, historial de ventas, actualización de precios.
- Dashboards analíticos: ventas por categoría, segmentación de clientes RFM.

2.3. Requisitos No Funcionales

Atributo	Métrica	Motor
Tiempo de respuesta transaccional	< 200 ms para consultas OLTP	PostgreSQL (Supabase)
Tiempo de respuesta analítico	< 500 ms para queries de catálogo	MongoDB (Atlas M0)
Volumen de datos	Soporte 99.441 pedidos, 32.951 productos	Ambos motores
Disponibilidad	>= 99% con plataformas free tier	Supabase + Atlas M0
Escalabilidad	Particionamiento horizontal en orders	PostgreSQL partitions
Consistencia	ACID para transacciones financieras	PostgreSQL

Flexibilidad de esquema	Atributos variables por tipo de producto	MongoDB
-------------------------	--	---------

2.4. Tabla de Volúmenes de Datos (Dataset Olist)

Entidad	Registros Actuales	Proyección 1 año	Motor Asignado
customers	96.096	~150.000	PostgreSQL
orders	99.441	~180.000	PostgreSQL (particionado)
order_items	112.650	~210.000	PostgreSQL
order_payments	103.886	~190.000	PostgreSQL
sellers	3.095	~6.000	PostgreSQL
products	32.951	~80.000	MongoDB (catálogo)
order_reviews	100.000	~220.000	MongoDB
geolocation	1.000.163 (dedup)	Sin cambio	PostgreSQL
user_behavior (eventos)	N/A	~500.000/mes	MongoDB (TTL 30d)
analytics_snapshots	N/A	~36 docs/año	MongoDB

3. Diseño Conceptual

3.1. Entidades del Dominio

Se identificaron 9 entidades principales del dominio de Ecommify a partir del análisis del dataset Olist y los requisitos funcionales del sistema. Cada entidad fue evaluada según su naturaleza (transaccional vs. analítica) para asignarla al motor de base de datos más adecuado.

Entidad	Descripción	Motor	Justificación
Customer	Usuario comprador con datos de contacto y dirección	PostgreSQL	Datos estructurados, requiere integridad referencial con orders
Order	Pedido con estado, fechas y relación con cliente	PostgreSQL	Núcleo transaccional, requiere ACID y particionamiento
OrderItem	Línea de pedido: producto, vendedor, precio, flete	PostgreSQL	Alta frecuencia de escritura, integridad con orders y products
OrderPayment	Pago con método, cuotas y valor	PostgreSQL	Datos financieros críticos, requiere consistencia ACID
Seller	Vendedor con datos geográficos y de contacto	PostgreSQL	Datos estables, requiere integridad con order_items
Product (transaccional)	Referencia de producto con stock y precio	PostgreSQL	Stock y precio requieren transacciones atómicas
Product Catalog	Catálogo con especificaciones variables	MongoDB	Esquema flexible por tipo de producto, atributos dinámicos
OrderReview	Reseña con texto, calificación y metadata	MongoDB	Datos semi-estructurados, consultas analíticas por sentimiento
UserBehavior	Eventos de navegación del usuario	MongoDB	Alta frecuencia de inserción, TTL de 30 días, schema flexible

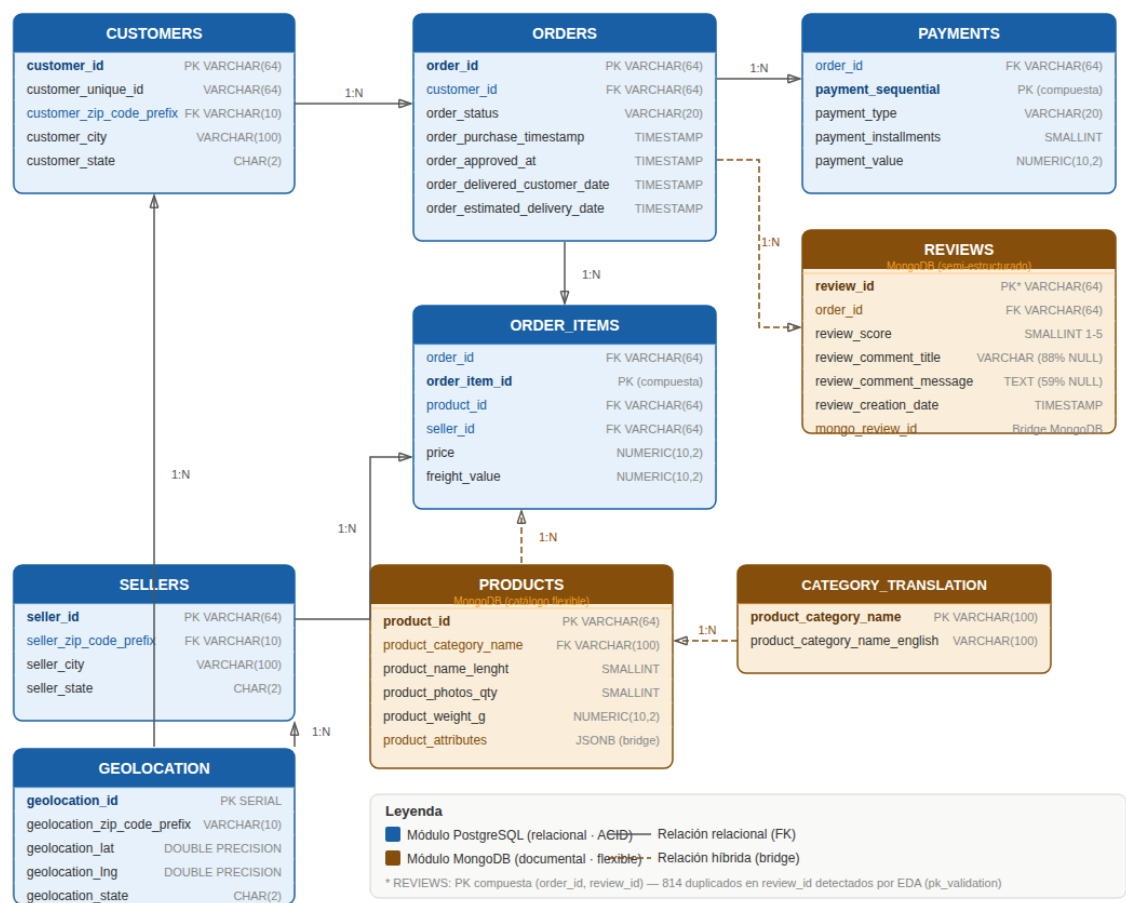


Figura 1. Diagrama Entidad-Relación (ER) del sistema Ecommify con 9 entidades, PKs, FKs y cardinalidades reales del EDA. Azul = PostgreSQL; Naranja = módulo híbrido MongoDB. (*) REVIEWS usa PK compuesta (order_id, review_id) por 814 duplicados detectados en review_id.

3.2. Relaciones y Cardinalidades

Las relaciones entre entidades determinan la estructura de llaves foráneas y la estrategia de join entre ambos motores:

Relación	Cardinalidad	Implementación
Customer → Order	1:N (un cliente tiene muchos pedidos)	FK customer_id en tabla orders
Order → OrderItem	1:N (un pedido tiene múltiples ítems)	FK order_id en tabla order_items
Order → OrderPayment	1:N (un pedido puede tener múltiples pagos parciales)	FK order_id en tabla order_payments
Order → OrderReview	1:1 (una review por pedido)	Campo order_id en colección MongoDB reviews
Seller → OrderItem	1:N (un vendedor tiene múltiples ítems)	FK seller_id en tabla order_items
Product → OrderItem	1:N (un producto aparece en múltiples ítems)	FK product_id en tabla order_items
Product PG → Product Mongo	1:1 (bridge via mongo_product_id)	Campo JSONB en products de PostgreSQL

3.3. Restricciones de Negocio Identificadas

- Un pedido no puede tener stock negativo: CHECK (quantity > 0) en order_items.
- Solo compradores con pedido entregado pueden publicar una reseña del producto.
- El stock en PostgreSQL debe decrementarse atómicamente al confirmar el pago (transacción).
- Una promoción no puede solaparse con otra para el mismo producto: EXCLUDE USING GIST con TSTZRANGE.
- El correo electrónico del cliente debe ser único: UNIQUE constraint en customers.email.
- El valor de pago debe ser positivo: CHECK (payment_value > 0) en order_payments.
- El estado del pedido solo puede tomar valores predefinidos: CHECK con lista de 8 estados válidos.

4. Diseño Lógico — Módulo PostgreSQL

4.1. Esquema Relacional Normalizado (3FN)

El módulo transaccional cumple la Tercera Forma Normal en todas sus tablas. La normalización fue aplicada sobre la tabla desnormalizada OrdersFull del dataset Olist, eliminando: grupos repetitivos (1FN), dependencias parciales de la clave primaria (2FN) y dependencias transitivas entre atributos no clave (3FN).

4.2. Tipos de Datos Avanzados Aplicados

La guía de la Unidad 2 exige la aplicación justificada de tipos avanzados. A continuación se documenta cada uno con su caso de uso en Ecommify:

Tipo Avanzado	Tabla.Columna	Justificación Técnica
JSONB	products.specifications	Atributos variables por categoría: laptops tienen RAM y almacenamiento; teléfonos tienen resolución y batería. JSONB es indexable con GIN, más eficiente que JSON para consultas frecuentes.
TEXT[]	products.photos	Una galería de fotos es una colección sin estructura interna que siempre se consume completa. Array evita tabla adicional photo_urls con JOIN innecesario.
TEXT[]	products.tags	Etiquetas para búsqueda full-text. Se consulta con operador ANY o @>. Evita tabla intermedia product_tags para caso de uso simple.
TEXT[]	sellers.notification_emails	Un vendedor puede tener múltiples correos de notificación. Select múltiple en frontend, almacenamiento como array en BD.
JSONB	sellers.business_hours	Horarios por día con estructura variable (algunos días cerrado, otros con múltiples turnos). JSONB flexible sin columnas NULL innecesarias.
TSTZRANGE	promotions.promotion_period	Período de vigencia de promoción. Permite operador NOW() <@ promotion_period y EXCLUDE GIST para evitar solapamiento entre promociones del mismo producto.
address_type (composite)	customers.shipping_address	Dirección de envío siempre se consulta/actualiza completa. Composite type agrupa lógicamente los campos sin crear tabla separada.

Ejemplo de consulta con tipos avanzados:

```
-- Buscar laptops con RAM >= 16GB usando JSONB
SELECT product_id, product_name, specifications->>'ram_gb' AS ram
FROM products
WHERE specifications->>'category' = 'laptop'
AND (specifications->>'ram_gb')::INT >= 16;
```

```
-- Promociones activas en este momento usando TSTZRANGE
SELECT product_id, discount_percent FROM promotions
WHERE NOW() <@ promotion_period;
```

4.3. Extensiones de PostgreSQL

Extensión	Estado	Caso de Uso en Ecommify
uuid-oss	Usar	Generación de UUIDs para IDs distribuidos sin colisión entre servicios
pg_trgm	Usar	Búsqueda de productos tolerante a errores tipográficos: "samsung" encuentra "samsung"
postgis	Evaluar	Cálculo de distancia vendedor-cliente para optimizar costos de envío según geolocation
pgcrypto	Usar	Hash seguro de contraseñas con bcrypt; cifrado de datos sensibles de pago
btree_gin	Usar	Índices compuestos eficientes en columnas JSONB y TEXT[] de products
pg_stat_statements	Usar	Monitoreo de queries lentas en producción para optimización continua
hstore	Descartar	Funcionalidad cubierta por JSONB con mejor soporte de tipos e indexación

4.4. Estrategia de Particionamiento

La tabla orders se particiona por RANGE sobre order_purchase_timestamp. Esta estrategia se justifica por el volumen del dataset (99.441 registros entre 2016-2018) y el patrón de acceso: las consultas operacionales acceden mayoritariamente a pedidos recientes (particiones calientes), mientras los reportes históricos acceden a particiones frías.

Partición	Rango	Tipo	Justificación
orders_2016_2017	2016-01-01 a 2018-01-01	Fría (cold)	Datos históricos, solo consultas analíticas ocasionales
orders_2018_q1	2018-01-01 a 2018-04-01	Semi-fría	Reportes trimestrales, baja frecuencia de acceso

orders_2018_q2	2018-04-01 a 2018-07-01	Semi-fría	Reportes trimestrales, baja frecuencia de acceso
orders_2018_q3	2018-07-01 a 2018-10-01	Caliente (hot)	Período de mayor actividad en el dataset Olist
orders_2018_q4	2018-10-01 a 2019-01-01	Caliente (hot)	Período más reciente, mayor frecuencia de consulta
orders_future	DEFAULT	Caliente (hot)	Recibe datos nuevos; se crea partición mensual programáticamente

4.5. Vistas Materializadas para Dashboards (OLAP)

Se diseñaron dos vistas materializadas para separar la carga OLAP de las operaciones OLTP. El refresh se programa semanalmente los domingos a las 2:00 AM para minimizar impacto en usuarios.

Vista Materializada	Propósito	Refresh	Índice Principal
mv_sales_by_category_monthly	Ventas totales, revenue e items por categoría y mes	Semanal (domingo 2am)	UNIQUE (year, month, category)
mv_customer_segments	Segmentación RFM: VIP, Regular, Occasional, New	Semanal (domingo 2am)	customer_segment, UNIQUE customer_id

4.6. Triggers de Auditoría

- fn_set_updated_at(): función reutilizable que actualiza el campo updated_at antes de cada UPDATE. Se aplica a: customers, sellers, products, orders, order_items, order_payments, order_reviews.
- fn_audit_order_status(): registra cada cambio de estado de un pedido en la tabla order_status_history con: order_id, old_status, new_status, changed_at, changed_by.

5. Diseño Lógico Preliminar — Módulo MongoDB

5.1. Colecciones Principales

Colección	Propósito	Patrón de Modelado	Por qué MongoDB y no PostgreSQL
products_catalog	Catálogo con especificaciones variables por categoría	Documento embebido	Esquema variable: laptops, teléfonos y TVs tienen atributos completamente diferentes. JSONB en PG sería un workaround; MongoDB es nativo.
reviews	Reseñas con texto libre, calificación y metadata	Documento autónomo + referencia a PG	Texto libre y metadata semi-estructurada. Análisis de sentimiento futuro. Alta variabilidad de campos.
user_behavior	Eventos de navegación con TTL 30 días	Insert-only + TTL index	Alta tasa de inserción (cientos por minuto). No se actualiza, solo se inserta y expira. PG no tiene TTL nativo.
analytics_snapshots	Snapshots mensuales de métricas precalculadas	Documento de agregación	Evita recalcular sobre 112k rows de order_items en cada consulta de dashboard. Complementa las MVs de PG.

5.2. Esquema de Documento — products_catalog

```
{
  "_id": ObjectId("..."),
  "postgres_product_id": "uuid-del-producto",
  "product_name": "Laptop Dell XPS 15",
  "category": "laptops",
  "price": 4500000,
  "specifications": {
    "ram_gb": 16, "storage_gb": 512,
    "processor": "Intel i7-12700H",
    "screen_inches": 15.6, "os": "Windows 11"
  },
  "photos": ["url1.jpg", "url2.jpg"],
  "avg_rating": 4.3, "total_reviews": 127,
  "seller_id": "uuid-vendedor",
  "updated_at": ISODate("2025-05-21T10:00:00Z")
}
```

5.3. Esquema de Documento — reviews

```

{
  "_id": ObjectId("..."),
  "postgres_order_id": "uuid-orden",
  "postgres_product_id": "uuid-producto",
  "customer_id": "uuid-cliente",
  "score": 4,
  "title": "Excelente producto",
  "comment": "Llegó en perfectas condiciones...",
  "photos": ["foto_review1.jpg"],
  "created_at": ISODate("2018-08-15T14:30:00Z"),
  "answer_timestamp": ISODate("2018-08-16T09:00:00Z"),
  "seller_response": "Gracias por su compra!"
}

```

5.4. Índices por Colección

Colección	Comando createIndex()	Justificación
products_catalog	{ "category": 1, "price": 1 }	Filtrado principal: por categoría y rango de precio
products_catalog	{ "product_name": "text" }	Búsqueda full-text por nombre de producto
products_catalog	{ "postgres_product_id": 1 }, unique:true	Bridge con PostgreSQL: lookup por ID de PG
reviews	{ "postgres_product_id": 1, "score": -1 }	Reviews por producto ordenadas por calificación
reviews	{ "customer_id": 1 }	Historial de reviews por cliente
user_behavior	{ "created_at": 1 }, expireAfterSeconds: 2592000	TTL de 30 días para limpieza automática
user_behavior	{ "customer_id": 1, "event_type": 1 }	Análisis de embudo de conversión por usuario
analytics_snapshots	{ "period": 1, "period_type": 1 }, unique:true	Lookup de snapshot por período

6. Decisiones Arquitectónicas Justificadas

6.1. Matriz de Decisión por Módulo/Entidad

Módulo	Entidad	Volumen	Consistencia	Estructura	Patrón Acceso	Motor
Usuarios	Customer	96k	ACID	Estructurado	Transaccional	PostgreSQL
Pedidos	Order	99k	ACID crítico	Estructurado	OLTP + particiones	PostgreSQL
Ítems	OrderItem	112k	ACID	Estructurado	Alta escritura	PostgreSQL
Pagos	OrderPayment	103k	ACID crítico	Estructurado	Financiero	PostgreSQL
Vendedores	Seller	3k	ACID	Estructurado	Lectura frecuente	PostgreSQL
Catálogo	Product Catalog	33k+	BASE (A+P)	Semi-estructurado	Búsqueda analítica	MongoDB
Reseñas	OrderReview	100k	BASE (A+P)	Semi-estructurado	Analítico/texto	MongoDB
Comportamiento	UserBehavior	500k/mes	BASE (A+P)	Flexible	Insert-only + TTL	MongoDB
Analítica	AnalyticsSnapshot	~36/año	BASE (A+P)	Flexible	Lectura dashboard	MongoDB

6.2. Análisis del Teorema CAP

El Teorema CAP establece que un sistema distribuido no puede garantizar simultáneamente Consistencia (C), Disponibilidad (A) y Tolerancia a particiones (P). La arquitectura de Ecommify aplica este principio diferenciando explícitamente entre módulos:

Módulo	Garantía CAP	Razón
Pagos y Pedidos (PostgreSQL)	C + P	Un cobro duplicado o un stock negativo son inaceptables. Se sacrifica disponibilidad ante partición de red.
Inventario (PostgreSQL)	C + P	La sobreventa genera pérdidas directas. Consistencia es no negociable.
Catálogo (MongoDB)	A + P	Mostrar un precio ligeramente desactualizado es aceptable. Disponibilidad mejora la experiencia de búsqueda.
Reseñas (MongoDB)	A + P	Una reseña con demora de segundos en aparecer es tolerable. Alta disponibilidad de lectura es prioritaria.

Comportamiento (MongoDB)	A + P	Eventos de navegación no críticos. Perder algunos eventos es aceptable; la disponibilidad de registro es clave.
--------------------------	-------	---

6.3. Trade-offs y Estrategias de Mitigación

Trade-off	Impacto	Estrategia de Mitigación
Consistencia entre motores	El stock en PG y el catálogo en Mongo pueden desincronizarse	ETL de sincronización programada cada 5 minutos vía script Python en Colab; campo mongo_product_id como bridge
Complejidad operativa	Operar dos motores aumenta la carga de mantenimiento	docker-compose local unifica ambos entornos; Supabase y Atlas M0 gestionan infra en producción
Consultas cross-motor	No se pueden hacer JOINS nativos entre PG y Mongo	Las consultas que cruzan motores se resuelven en la capa de aplicación (backend) en dos pasos
Límites del free tier	Supabase: 500 MB, Atlas M0: 512 MB	El dataset Olist completo cabe en ambos límites; se documenta plan de upgrade si el proyecto crece

6.4. Flujo de Sincronización ETL

La sincronización entre PostgreSQL y MongoDB se implementa mediante un script Python ejecutado en Google Colab con los siguientes 6 pasos:

- Paso 1 — Extracción: leer productos modificados en PostgreSQL en los últimos 5 minutos (campo updated_at).
- Paso 2 — Transformación: construir documento MongoDB enriquecido con especificaciones JSONB y URLs de fotos.
- Paso 3 — Carga: upsert en MongoDB usando postgres_product_id como clave de sincronización.
- Paso 4 — Verificación: comparar conteo de registros modificados vs. documentos actualizados.
- Paso 5 — Log: registrar resultado de la sincronización en tabla etl_log de PostgreSQL.
- Paso 6 — Alerta: si hay discrepancia > 1%, enviar notificación al email del equipo.

7. Estrategia de Mantenimiento Operativo

7.1. Jobs Programados

Job	Frecuencia	Herramienta	Propósito
VACUUM ANALYZE en orders, order_items, order_payments	Diario (3:00 AM)	pg_cron (Supabase)	Recuperar espacio de tuplas muertas y actualizar estadísticas del planificador
REFRESH MATERIALIZED VIEW mv_sales_by_category_monthly	Semanal (domingo 2:00 AM)	pg_cron	Actualizar datos de dashboard de ventas por categoría
REFRESH MATERIALIZED VIEW mv_customer_segments	Semanal (domingo 2:30 AM)	pg_cron	Actualizar segmentación RFM de clientes
Crear partición del mes siguiente en orders	Mensual (día 1, 1:00 AM)	pg_cron + función PL/pgSQL	Prevenir que datos nuevos caigan en partición DEFAULT
REINDEX CONCURRENTLY en índices críticos	Mensual (domingo 4:00 AM)	pg_cron	Mantener índices compactos y eficientes en tablas de alto volumen
Limpieza TTL user_behavior en MongoDB	Automático (motor TTL index)	MongoDB Atlas	Eliminar eventos de navegación con más de 30 días automáticamente

7.2. Métricas de Monitoreo

Métricas OLTP (PostgreSQL):

- Cache hit ratio (objetivo > 99%): `SELECT sum(heap_blks_hit) / (sum(heap_blks_hit) + sum(heap_blks_read)) FROM pg_statio_user_tables;`
- Conexiones activas vs. máximo configurado en Supabase free tier (máx. 60).
- Top 10 queries más lentas vía `pg_stat_statements`: `ORDER BY total_exec_time DESC LIMIT 10.`
- Tamaño de cada partición de orders: `SELECT partition, pg_size_pretty(size) FROM...`

Métricas OLAP (MongoDB):

- Tiempo del último refresh de vistas materializadas vs. umbral de 15 minutos.
- Uso de índices en Atlas: `serverStatus().indexCounters` para detectar índices no utilizados.
- Tamaño de colección user_behavior (debe estabilizarse con TTL activo).

7.3. Plan de Escalamiento

Umbral	Acción	Motor
Supabase > 400 MB (80% del free tier)	Migrar a plan Pro (\$25/mes) o limpiar datos históricos de geolocation (1M registros)	PostgreSQL
Atlas M0 > 400 MB (80% del free tier)	Migrar a M2 (\$9/mes) o archivar analytics_snapshots antiguos	MongoDB
Tiempo de respuesta OLTP > 200 ms sostenido	Revisar plan de índices, activar connection pooling (PgBouncer en Supabase)	PostgreSQL
Partición DEFAULT supera 10k registros	Ejecutar creación de partición mensual manualmente antes del job programado	PostgreSQL

8. Anexos

Anexo A — Diccionario de Datos (Tablas PostgreSQL)

Tabla	Columna	Tipo	Restricción	Descripción
customers	customer_id	UUID	PK	Identificador único del cliente generado con uuid-oss
customers	customer_email	VARCHAR(255)	UNIQUE, NOT NULL	Correo electrónico principal del cliente
customers	shipping_address	address_type	NULLABLE	Dirección de envío usando composite type
orders	order_id	UUID	PK	Identificador único del pedido
orders	customer_id	UUID	FK → customers	Referencia al cliente que realizó el pedido
orders	order_status	VARCHAR(20)	CHECK (8 estados)	Estado: created, approved, invoiced, processing, shipped, delivered, canceled, unavailable
orders	order_purchase_timestamp	TIMESTAMPTZ	NOT NULL, PARTITION KEY	Fecha/hora de creación del pedido (clave de particionamiento)
order_items	order_item_id	SMALLINT	PK compuesta con order_id	Número de ítem dentro del pedido
order_items	item_total_value	NUMERIC(10,2)	GENERATED STORED	Columna calculada: price + freight_value
products	specifications	JSONB	DEFAULT '{}'	Atributos variables por categoría de producto
products	photos	TEXT[]	DEFAULT '{}'	URLs de fotos del producto
promotions	promotion_period	TSTZRANGE	NOT NULL, EXCLUDE GIST	Período de vigencia de la promoción

Anexo B — Referencias a Scripts SQL

Los scripts SQL completos se encuentran en el repositorio GitHub del proyecto bajo la ruta `database/postgresql/schema/`:

- `00_base_schema.sql` — Esquema base: tablas principales, constraints e índices fundamentales

- 01_extensions.sql — Habilitación de extensiones con justificación
- 02_advanced_types.sql — Tipos avanzados: composite types, columnas JSONB/ARRAY/TSTZRANGE
- 03_partitioning.sql — Tabla orders particionada por fecha con índices locales
- 04_materialized_views.sql — Vistas materializadas para dashboards OLAP
- 05_triggers.sql — Triggers updated_at y auditoría de cambios de estado
- 06_maintenance_jobs.sql — Jobs programados de mantenimiento y queries de monitoreo

Anexo C — Referencias a Esquemas MongoDB

Los esquemas de colecciones MongoDB se encuentran en `database/mongodb/schema/collections_schema.json` con: estructura de documento, índices con comandos `createIndex()` y justificación de diseño para cada colección.

Referencias

MongoDB, Inc. (2025). *Data modeling introduction*. MongoDB Manual.

<https://www.mongodb.com/docs/manual/core/data-modeling-introduction/>

PostgreSQL Global Development Group. (2025). *Data types*. PostgreSQL 16 Documentation.

<https://www.postgresql.org/docs/16/datatype.html>

PostgreSQL Global Development Group. (2025). *Server programming: Extending SQL*.

PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/extend.html>

Brewer, E. (2012). CAP twelve years later: How the rules have changed. *IEEE Computer*. 45(2), 23–29.